

Gaussian Belief Propagation-based Task Allocation (GBPTA) [Revised]

Anonymous Authors

Abstract—Multi-robot systems are used in various fields such as autonomous transportation and delivery. A key component in enabling these systems is task allocation, which becomes significantly more complex when coordinating multiple robots. For solving the task allocation problem, the challenges extend beyond those faced by a single robot, requiring robust coordination strategies. Moreover, a critical factor in evaluating multi-robot systems is their ability to operate reliably in real-world, dynamic environments. In many scenarios, ground-truth information about robots and tasks may be unavailable, demanding methods that can function with noisy observations and handle substantial uncertainty.

In this paper, we propose GBPTA, a fully distributed framework for multi-robot multi-task allocation problem, using Gaussian Belief Propagation for solving the one-to-one assignment problem using an end-to-end factor-graph-based approach, where all steps are represented as factors. The proposed framework is applied to two scenarios: multi-robot pursuit-evasion and package pickup and delivery. GBPTA can operate in different environments and under varying communication radii. Moreover, the proposed approach can handle problems in which multiple features must be considered for task allocation. Through various simulations and hardware experiments, we show that GBPTA achieves near-optimal task assignments. GBPTA improves assignment utility, defined as the negative total distance between robots and their assigned tasks, by 23.95% over distributed Hungarian and reduces makespan by 71.57% over DANCeRS.

I. INTRODUCTION

Multi-robot systems (MRS) are vital for applications such as search and rescue [1] and package delivery [2], requiring efficient collaboration to solve complex and coordinated tasks. Unlike single-agent systems, MRS must manage coordinated decision-making, communication constraints, and inter-robot dependencies in dynamic and uncertain environments [3].

While centralized and decentralized coordination methods exist, fully distributed approaches offer scalability and resilience to single-point failures [4]. However, current distributed task allocation methods often struggle with uncertainty. Further, in evolving environments, robots must jointly estimate task dynamics and plan feasible trajectories, motivating a unified framework that integrates these components into the allocation process.

This paper introduces GBPTA, a fully distributed multi-robot task allocation framework based on Gaussian Belief Propagation (GBP) for solving one-to-one assignment. GBP is useful for task allocation because it provides a distributed, scalable, and uncertainty-aware mechanism for reaching agreement among multiple robots. By utilizing a factor graph representation, GBPTA enables coordinated decision-making under uncertainty, incorporating non-linear soft-assignment factors with constraints such as utility, regret, and collision penalties.

Our key contributions include: 1) A GBP-based factor graph framework that treats state estimation and trajectory

planning as auxiliary features to enhance allocation robustness. 2) Demonstration of the framework’s generalizability across diverse scenarios, specifically pursuit–evasion (PE) and Multi-Robot Pickup and Delivery (MRPD) problems.

3) A fully distributed approach that reduces average travel distance compared to existing decentralized baselines while maintaining resilience to sensor noise and system failures.

The paper is organized as follows: Sec. II reviews related work, Sec. III defines the problem, Sec. IV and V detail the framework and its applications, and Sec. VI and VII present results and conclusions.

II. RELATED WORKS

When solving a problem with MRS, their coordination is a key step. Multi-robot coordination typically relies on centralized, decentralized, or distributed architectures to manage task allocation and communication [5]. Centralized task allocation methods, such as the Hungarian algorithm [6], PDAGA [7], Hybrid SUSP [8], MRMTA [9], and Sequential Greedy [10] are widely utilized, yet they often face challenges regarding scalability and single-point-of-failure vulnerabilities. In contrast, distributed systems partition computation among individual nodes, enhancing both scalability and robustness against central node failures. Notable distributed approaches include: Distributed Hungarian algorithms, which parallelize assignment logic [11]; Consensus-Based Bundle Algorithm (CBBA), which employs market-based task selection paired with consensus protocols for conflict resolution [12]; The framework by Osooli et al., designed for heterogeneous search-and-rescue teams to optimize task distribution [9]; and ADMM-based optimization, which decomposes global objectives into parallel subproblems coordinated via shared variables [13].

MRPD and PE are important application of multi-robot systems, where a team of robots must collaboratively execute transportation tasks while avoiding collisions. Several approaches have been proposed to address task allocation and coordination challenges in MRPD. DD-MAPD introduced a framework for multi-robot shelf rearrangement [14], while Camisa *et al.* proposed a distributed resource allocation method for pickup-and-delivery vehicle routing problems [15]. The Marginal-cost Based Auction Algorithm (MAA) further addressed large-scale package transportation using a distributed task allocation strategy [16]. GATAR proposed a Graph-based approach to model the relationships and interactions among agents through structured connectivity, enabling coordinated pursuit and decision-making. SUSP [8], provides an adaptive coordination strategy from data and interaction with the environment.

Gaussian Belief Propagation (GBP) offers a robust alternative

for distributed inference. In GBP, factor graphs represent variables (unknowns) and factors (constraints) as a bipartite model [17]. GBP involves iterative message passing to update variable beliefs. For Gaussian factors, the distribution is:

$$f(x; z) \propto \exp\left(-\frac{1}{2}(z - h(x))^T \Omega_s(z - h(x))\right), \quad (1)$$

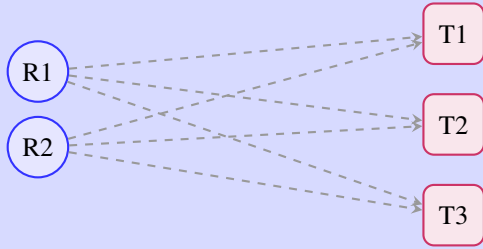
where z is the observation, $h(x)$ is the measurement function, and $\Omega_s = \Sigma^{-1}$ is the information matrix weighting the residual based on measurement uncertainty Σ . **GBP has been used for various applications such as localization [18], and Planning [19]. DANCeRS introduced a distributed algorithm for consensus negotiation through factor graphs and occupancy weighting [20].**

III. PROBLEM STATEMENT

In this section, the problem statement for multi-robot task allocation (MRTA) is presented. We consider a MRTA problem consisting of a set of N_p robots and a set of N_q tasks, where $N_p \neq N_q$ represents an unbalanced assignment scenario.

An example for 2 robots and 3 tasks:

Set of Robots ($N_p = 2$) Set of Tasks ($N_q = 3$)



The state of each task is represented by a vector of horizontally concatenated scalar values comprising relevant attributes; for example, the distance between each robot and the task, the package weight in **MRPD** scenarios, or the robots' remaining energy. These values form the cost vector $\mathbf{c}_t^i$ shown in Eq. (2).

$$\mathbf{c}_t^i = [c_t^{i1}, \dots, c_t^{iN_q}]^T, \mathbf{c}_t^i \in \mathbb{R}^{N_q \times 1}, i \in \mathbb{N}, i \geq 1, \quad (2)$$

where scalar c_t^{ij} denotes the state of task j with respect to robot i at time t .

Example: Cost Vectors for Robots $i = 1$ and $i = 2$ ($N_q = 3$ tasks)

In this scenario, each robot evaluates its specific relationship to the three tasks (e.g., distance).

Robot 1 ($i = 1$):

- c_t^{11} (Task 1): 5.2 m
- c_t^{12} (Task 2): 12.0 m
- c_t^{13} (Task 3): 3.5 m

$$\mathbf{c}_t^1 = \begin{bmatrix} 5.2 \\ 12.0 \\ 3.5 \end{bmatrix}$$

Robot 2 ($i = 2$):

- c_t^{21} (Task 1): 8.4 m
- c_t^{22} (Task 2): 2.1 m
- c_t^{23} (Task 3): 10.9 m

$$\mathbf{c}_t^2 = \begin{bmatrix} 8.4 \\ 2.1 \\ 10.9 \end{bmatrix}$$

Each vector $\mathbf{c}_t^i \in \mathbb{R}^{3 \times 1}$ captures the unique costs from that robot's perspective.

Assume, $s_t^{ij} \in \{0, 1\}$ is a binary variable indicating whether the j^{th} task is selected by i^{th} robot at time t , and $\mathbf{s}_t^i$ is a vector of such binary variables representing the task selection status of robot i across all tasks, at time t :

$$\mathbf{s}_t^i = [s_t^{i1}, \dots, s_t^{iN_q}]^T, \mathbf{s}_t^i \in \mathbb{R}^{N_q \times 1}, i \in \mathbb{N}, i \geq 1. \quad (3)$$

Example: Selection Vectors for Robots $i = 1$ and $i = 2$ ($N_q = 3$ tasks)

Suppose at time t , Robot 1 is assigned to Task 3, and Robot 2 is assigned to Task 2.

Robot 1 ($i = 1$):

- $s_t^{11} = 0$
- $s_t^{12} = 0$
- $s_t^{13} = 1$

$$\mathbf{s}_t^1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Robot 2 ($i = 2$):

- $s_t^{21} = 0$
- $s_t^{22} = 1$
- $s_t^{23} = 0$

$$\mathbf{s}_t^2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Here, each $\mathbf{s}_t^i$ is a one-hot vector if a robot can only select one task at a time.

While the final assignment is discrete, we define a continuous assignment belief vector $\mathbf{a}_t^i \in [0, 1]^{N_q \times 1}$ for each robot i . Each element a_t^{ik} represents the current estimated probability or preference of robot i being assigned to task k . The relationship between this continuous belief and the discrete assignment s_t^{ij} is governed by a selection threshold τ :

$$s_t^{ij} = 1 \quad \text{if} \quad a_t^{ij} > \tau. \quad (4)$$

This formulation allows the assignment problem to be solved using differentiable optimization techniques while ensuring a clear path to a discrete multi-robot coordination.

Example: Mapping Belief $\mathbf{a}_t^1$ to Selection $\mathbf{s}_t^1$ with $\tau = 0.5$

Belief Vector $\mathbf{a}_t^1$:

- $a_t^{11} = 0.12$
- $a_t^{12} = 0.38$
- $a_t^{13} = 0.89$

$$\mathbf{a}_t^1 = \begin{bmatrix} 0.12 \\ 0.38 \\ 0.89 \end{bmatrix}$$

Resulting Selection $\mathbf{s}_t^1$:

- $s_t^{11} = 0$
- $s_t^{12} = 0$
- $s_t^{13} = 1$

$$\mathbf{s}_t^1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

This ensures that the continuous optimization for Task 3 has reached a high enough confidence to trigger a discrete assignment.

The objective of MRTA is to allocate tasks to robots such that the total execution cost is minimized. This is formulated as the following optimization problem:

$$\begin{aligned} \min_{\mathbf{s}_t^i} \quad & \sum_{i=1}^{N_p} (\mathbf{c}_t^i)^T \mathbf{s}_t^i \\ \text{s.t.} \quad & \sum_{j=1}^{N_q} s_t^{ij} \leq 1, \quad \forall i \in \{1, \dots, N_p\} \\ & \sum_{i=1}^{N_p} s_t^{ij} \leq 1, \quad \forall j \in \{1, \dots, N_q\} \\ & s_t^{ij} \in \{0, 1\} \end{aligned} \quad (5)$$

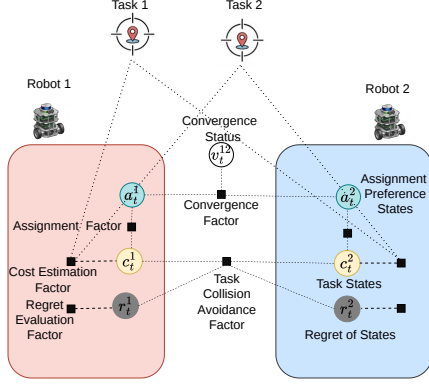


Fig. 1. General overview of the GBPTA framework. The proposed method operates on a factor graph comprising variable nodes (circles) and factor nodes (solid squares). Factors represent constraints between variables, ensuring a unique task-to-robot assignment where each task is allocated to exactly one robot and vice versa. This process is executed in a fully distributed manner. As demonstrated in our experiments, this assignment graph can be augmented with additional factor graphs for motion planning or state estimation to support robotic missions.

The inequalities in Eq. (5) account for unbalanced scenarios where the number of robots N_p and tasks N_q differ, thereby accommodating idle robots or unassigned tasks. These constraints enforce a partial matching between the two sets, ensuring that each task is allocated to at most one robot and each robot is assigned no more than a single task at time t .

Example: Objective and Constraints for $N_p = 2$ and $N_q = 3$
Using the costs and selections from previous examples:

Robot 1 ($i = 1$):

Robot 2 ($i = 2$):

$$\mathbf{c}_t^1 = \begin{bmatrix} 5.2 \\ 12.0 \\ 3.5 \end{bmatrix}, \mathbf{s}_t^1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad \mathbf{c}_t^2 = \begin{bmatrix} 8.4 \\ 2.1 \\ 10.9 \end{bmatrix}, \mathbf{s}_t^2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Cost contribution: 3.5

Cost contribution: 2.1

1. Total Cost Calculation:

$$\text{Total Cost} = (\mathbf{c}_t^1)^\top \mathbf{s}_t^1 + (\mathbf{c}_t^2)^\top \mathbf{s}_t^2 = 3.5 + 2.1 = 5.6$$

2. Constraint Check:

- Robot 1: $\sum s^{1j} = 1 \leq 1$ (Assigned to Task3) ✓
- Robot 2: $\sum s^{2j} = 1 \leq 1$ (Assigned to Task2) ✓
- Tasks: Task1 is unassigned ($\sum s^{i1} = 0 \leq 1$), Task2/Task3 each have one robot. ✓

IV. PROPOSED METHOD

In this section, the task allocation problem is formulated as a factor graph and solved using GBP. As illustrated in Fig. 1, the formulation introduces five factors that model (1) task evaluation, (2) regret evaluation, (3) task collision avoidance, (4) assignment, and (5) convergence. The following subsections describe each factor in detail.

A. Task Evaluation Factor

The task evaluation factor, denoted by $f_t^{c_i}(\mathbf{c}_t^i)$, assesses the state of tasks for robot i at time t . It represents the robot's be-

lief of the cost associated with each potential task assignment. The observation model for this factor is:

$$h^c(\mathbf{c}_t^i) = \mathbf{c}_t^i. \quad (6)$$

For this factor, measurement $\mathbf{z}_t^{c_i}$ represents the observed state of tasks for robot i . The likelihood of the state given the observation is then modeled by the Gaussian distribution in Eq. (1), where the residual $\mathbf{z}_t^{c_i} - h^c(\mathbf{c}_t^i)$ is weighted by the information matrix Ω_s to account for observation uncertainty.

Example: Observed vs. Predicted Task Costs ($i = 1$)

Continuing with Robot 1 and $N_q = 3$ tasks, we compare the internal state $\mathbf{c}_t^1$ with an observation $\mathbf{z}_t^{c_1}$ affected by sensor noise.

Internal State $h^c(\mathbf{c}_t^1)$:

Observation $\mathbf{z}_t^{c_1}$:

$$\mathbf{c}_t^1 = \begin{bmatrix} 5.2 \\ 12.0 \\ 3.5 \end{bmatrix} \quad \mathbf{z}_t^{c_1} = \begin{bmatrix} 5.3 \\ 11.8 \\ 3.6 \end{bmatrix}$$

(The robot's current belief) (Noisy sensor measurement)

Residual ($\mathbf{z}_t^{c_1} - h^c(\mathbf{c}_t^1)$):

$$\text{Residual} = \begin{bmatrix} 5.3 - 5.2 \\ 11.8 - 12.0 \\ 3.6 - 3.5 \end{bmatrix} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.1 \end{bmatrix}$$

The information matrix Ω_s will weight this residual to determine how much the robot should trust this new observation versus its prior belief.

B. Regret Evaluation Factor

The regret factor, $f_t^{r_i}(\mathbf{r}_t^i)$, measures the relative disadvantage, i.e., the opportunity cost, of assigning specific tasks to a robot. It evaluates the benefit of a particular task by considering alternative selections and quantifying the corresponding regret. The observation model for this factor is defined as follows:

$$h^r(\mathbf{r}_t^i) = \mathbf{r}_t^i, \quad (7)$$

where $\mathbf{r}_t^i$ is the regret vector for robot i at time t . Each element of the vector represents the associated regret of performing a specific task. The measurement $\mathbf{z}_t^{r_i}$ defines the regret. Each element $z_t^{r_{ij}}$ of the vector $\mathbf{z}_t^{r_i}$ represents the regret associated with robot i selecting task j , computed as:

$$z_t^{r_{ij}} = - \sum_{k \neq j} c_t^{ik}, \quad (8)$$

where c_t^{ik} represents the cost of alternative tasks. The belief and residual is subsequently computed using Eq. (1).

Example: Calculating Regret $\mathbf{z}_t^{r_1}$ for Robot 1 ($N_q = 3$)
Recall Robot 1's costs: $c_t^{11} = 5.2$, $c_t^{12} = 12.0$, $c_t^{13} = 3.5$. The regret for each task is the negative sum of the *other* available task costs:

- Regret for Task 1 ($j = 1$):
 $z_t^{r_{11}} = -(c_t^{12} + c_t^{13}) = -(12.0 + 3.5) = -15.5$
- Regret for Task 2 ($j = 2$):
 $z_t^{r_{12}} = -(c_t^{11} + c_t^{13}) = -(5.2 + 3.5) = -8.7$
- Regret for Task 3 ($j = 3$):
 $z_t^{r_{13}} = -(c_t^{11} + c_t^{12}) = -(5.2 + 12.0) = -17.2$

The resulting measurement vector is:

$$\mathbf{z}_t^{r_1} = \begin{bmatrix} -15.5 \\ -8.7 \\ -17.2 \end{bmatrix}$$

Selecting Task 3 (the lowest cost task) results in the highest absolute regret (-17.2), reflecting the significant loss of opportunity if the other, more expensive tasks were chosen instead.

C. Task Collision Avoidance Factor

To prevent multiple robots from selecting the same task, a task collision avoidance factor is introduced. This pairwise factor, denoted by $f_t^{ctij}(\mathbf{c}_t^i, \mathbf{c}_t^j, \mathbf{r}_t^i, \mathbf{r}_t^j)$, considers the task states and associated regrets for both robot i and robot j . Let $\mathbf{u}_t^\ell = \mathbf{c}_t^\ell + \mathbf{r}_t^\ell$ denote the combined utility vector, i.e., the sum of the state and regret of tasks, for robot $\ell \in \{i, j\}$, representing the total weighted cost of each task at time t .

The task collision avoidance factor works by penalizing configurations where two robots, i and j , have a high probability of being assigned to the same task. By applying the softmax function [21] to the combined utility $\mathbf{u}_t^\ell$, the factor generates a probability distribution over all possible tasks. If both robots express a high preference for the same task k , the factor increases the local cost, pushing one of the robots toward its second-best alternative.

Softmin allows the model to maintain differentiability and enables probabilistic task assignment, which is needed for gradient-based optimization and robust decision-making under uncertainty. Given a vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top$, the softmax function $\text{SM}(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is defined as [21]:

$$\text{SM}(\mathbf{x})_i = \frac{e^{-x_i}}{\sum_{j=1}^n e^{-x_j}}. \quad (9)$$

The softmax function transforms the utility vector $\mathbf{u}_t^\ell$ into a probability distribution π_t^ℓ , where each element $\pi_t^{\ell k}$ represents the probability of robot ℓ selecting task k .

The softmax function assigns higher probability to tasks with lower combined state and regret, resulting in task selection probabilities π_t^i and π_t^j for robots i and j , respectively.

$$\pi_t^\ell = \text{SM}(\mathbf{u}_t^\ell), \quad \ell \in \{i, j\}. \quad (10)$$

To check how likely two robots will select the same task, the cosine similarity of their utility distributions is calculated:

$$\kappa_t^{ij} = (\pi_t^i)^\top \pi_t^j. \quad (11)$$

The match score κ_t^{ij} acts as a scalar activation level for the collision factor; it remains near zero if the robots prefer different tasks and approaches one if their preferences conflict.

To softly allocate penalty between the two robots, we define element-wise softmax weights as:

$$\mathbf{w}_t^\ell = \kappa_t^{ij} \cdot \frac{e^{\beta \mathbf{u}_t^\ell}}{e^{\beta \mathbf{u}_t^i} + e^{\beta \mathbf{u}_t^j}}, \quad \ell \in \{i, j\}, \quad (12)$$

where β controls the smoothness of the approximation. As $\beta \rightarrow \infty$, the function approximates a hard decision. Finally, the task collision avoidance factor is implemented via the observation function $h_t^{ctij}(\cdot)$, which relates the current task costs and regrets to a penalty-based correction:

$$h_t^{ctij}(\mathbf{c}_t^i, \mathbf{c}_t^j, \mathbf{r}_t^i, \mathbf{r}_t^j) = p \cdot \begin{bmatrix} \mathbf{w}_t^i \\ \mathbf{w}_t^j \end{bmatrix} - \begin{bmatrix} \mathbf{c}_t^i \\ \mathbf{c}_t^j \end{bmatrix}, \quad (13)$$

where p is a scaling constant that determines the strength of the collision penalty.

The measurement vector $\mathbf{z}_t^{ctij}$ is defined as a zero vector, representing a target state with zero assignment conflict. By minimizing the difference between this zero-target and the observation function in (13), the framework forces the robots to negotiate and diverge from shared task preferences.

Example: Utility $\mathbf{u}_t^\ell$, Softmin π_t^ℓ , and Match Score κ_t^{12}
Using previous values for $i = 1$ and $j = 2$ with $N_q = 3$ tasks. Suppose the combined utility (Cost + Regret) results in:
Robot 1 ($i = 1$):

$$\mathbf{u}_t^1 = \begin{bmatrix} -10.3 \\ -3.5 \\ -13.7 \end{bmatrix} \xrightarrow{\text{SM}} \pi_t^1 \approx \begin{bmatrix} 0.03 \\ 0.00 \\ \mathbf{0.97} \end{bmatrix}$$

(Strongly prefers Task 3) Robot 2 ($j = 2$):

$$\mathbf{u}_t^2 = \begin{bmatrix} -1.2 \\ -12.4 \\ -3.5 \end{bmatrix} \xrightarrow{\text{SM}} \pi_t^2 \approx \begin{bmatrix} 0.00 \\ \mathbf{0.99} \\ 0.01 \end{bmatrix}$$

(Strongly prefers Task 2)

1. Match Score (Conflict Check):

$$\kappa_t^{12} = (\pi_t^1)^\top \pi_t^2$$

$$\kappa_t^{12} \approx (0.03)(0.0) + (0.0)(0.99) + (0.97)(0.01) = 0.0097$$

The resulting score is near 0. Both robots prefer different tasks (T3 vs T2), so the collision penalty is negligible.

2. Conflict Scenario: If π_t^2 also had 0.97 for Task 3, then $\kappa_t^{12} \approx 0.94$. The factor would then apply the penalty weight $\mathbf{w}_t^\ell$ to push one robot toward its second-best choice.

D. Assignment Factor

The assignment factor is responsible for selecting the task with the minimum state, where the state is the cost vector $\mathbf{c}_t^i$ (as defined in Eq. (2)). Since the discrete selection process is non-differentiable, we utilize a softmax-based observation model to provide a smooth approximation:

$$h^a(\mathbf{c}_t^i, \mathbf{a}_t^i) = \psi(\mathbf{c}_t^i) - \mathbf{a}_t^i, \quad (14)$$

where $\mathbf{a}_t^i$ represents the assignment belief vector for robot i . Initially, the assignment belief vector $\mathbf{a}_0^i$ is set to a uniform

distribution, where $a_0^{ik} = 1/N_q, \forall k \in \{1, \dots, N_q\}$, representing an undecided state where all tasks are equally likely to be assigned. The term $\psi(\mathbf{c}_t^i)$ is the softmax transformation defined as:

$$\psi^k = \frac{e^{-\beta c_t^{ik}}}{\sum_{j=1}^{N_q} e^{-\beta c_t^{ij}}}, \quad (15)$$

where ψ^k denotes the k -th component of the vector ψ , representing the probability that task k is the optimal choice for robot i . The measurement vector $\mathbf{z}_t^i$ is defined as a zero vector. This ensures that the residual is minimized when the robot's assignment belief $\mathbf{a}_t^i$ aligns perfectly with the differentiable task selection $\psi(\mathbf{c}_t^i)$.

To extract a discrete assignment from this probabilistic framework, a threshold τ is applied to the assignment belief. If the preference for a specific task j exceeds this threshold (i.e., $a_t^{ij} > \tau$), the task is considered selected, and we set $s_t^{ij} = 1$.

Example: Evolution from Initial $\mathbf{a}_0^1$ to Assignment $\mathbf{s}_t^1$
Using Robot 1's costs: $\mathbf{c}_t^1 = [5.2, 12.0, 3.5]^\top$ and $\beta = 1.0$.

1. Initial State (Undecided): At $t = 0$, the belief is uniform across $N_q = 3$ tasks:

$$\mathbf{a}_0^1 = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} \approx \begin{bmatrix} 0.33 \\ 0.33 \\ 0.33 \end{bmatrix}$$

2. Softmin Target $\psi(\mathbf{c}_t^1)$: The factor calculates the probability of each task being optimal based on low cost:

$$\psi(\mathbf{c}_t^1) \approx \begin{bmatrix} 0.15 \\ 0.00 \\ 0.85 \end{bmatrix} \quad (\text{Task 3 has highest probability})$$

3. Convergence and Selection ($\tau = 0.5$): As the optimization minimizes the residual h^a , the belief $\mathbf{a}_t^1$ moves toward ψ . Once $a_t^{13} > 0.5$:

$$\mathbf{s}_t^1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

E. Convergence Factor

The convergence factor monitors whether the global assignment constraints defined in Eq. (5) are satisfied across the entire swarm. It provides a differentiable measure of the total overlap between the assignment belief vectors of all robot pairs. The measurement function for this factor is:

$$h^{vij}(\mathbf{a}_t^i, \mathbf{a}_t^j) = -v_t^{ij} + (\mathbf{a}_t^i)^\top \mathbf{a}_t^j, \quad (16)$$

where v_t^{ij} is a scalar variable representing the **conflict between robots i and j and each of them keeps a copy of that..** The $(\mathbf{a}_t^i)^\top \mathbf{a}_t^j$ represents the actual conflict and in a perfect assignment, it should be **less than ϵ** , indicating orthogonality. Optimization pushes v_t^{ij} to accurately reflect the sum of the **overlap between robot i and robot j** . The observation for this factor, $\mathbf{z}_t^{vij}$, is set to zero to drive the swarm toward a state of mutual orthogonality. The optimization pushes the assignment vectors $\mathbf{a}$ to become orthogonal (zero overlap) so they can satisfy the $\mathbf{z}_t^{vij}$ goal.

If the convergence measure satisfies $v_t^{ij} \leq \epsilon$ for a predefined tolerance $\epsilon > 0$, the task assignment preference vectors are considered approximately orthogonal **between robots i and j . If all convergence status variables are less than or equal to ϵ ($v_t^{ij} \leq \epsilon, \forall i, j \in N_p$), the method is considered to have converged.** This state implies minimal pairwise overlap, ensuring that robots have reached a consensus on distinct task selections and that the constraints in Eq. (5) are satisfied.

Example: Global Conflict v_t and Orthogonality Check

Consider our two robots with their belief vectors $\mathbf{a}_t^1$ and $\mathbf{a}_t^2$.

1. High Conflict State (Both prefer Task 3):

$$\mathbf{a}_t^1 = \begin{bmatrix} 0.1 \\ 0.1 \\ 0.8 \end{bmatrix}, \quad \mathbf{a}_t^2 = \begin{bmatrix} 0.0 \\ 0.2 \\ 0.8 \end{bmatrix}$$

Actual Conflict = $(\mathbf{a}_t^1)^\top \mathbf{a}_t^2 = (0.1)(0) + (0.1)(0.2) + (0.8)(0.8) = 0.66$

Status: $v_t^{12} \approx 0.66 > \epsilon$. Constraints are violated; robots are fighting for Task 3.

2. Converged State (Orthogonal/Distinct):

$$\mathbf{a}_t^1 = \begin{bmatrix} 0.1 \\ 0.1 \\ 0.8 \end{bmatrix}, \quad \mathbf{a}_t^2 = \begin{bmatrix} 0.1 \\ 0.8 \\ 0.1 \end{bmatrix}$$

Actual Conflict = $(0.1)(0.1) + (0.1)(0.8) + (0.8)(0.1) = 0.17$

Status: As $v_t^{12} \rightarrow 0$, the vectors become orthogonal. Once $v_t^{12} \leq \epsilon$, the swarm has reached a consensus on distinct tasks.

F. Distributed GBP Solver

Given the factor graph formulation described above, the optimal task assignment is obtained by minimizing the total energy of the system via GBP. Unlike centralized approaches, this framework allows each robot to locally compute its assignment belief $\mathbf{a}_t^i$ and iteratively negotiate with neighbors to resolve conflicts. The distributed solver proceeds as follows:

Step 1: At time step t , each robot i observes the environment to compute the cost vector $\mathbf{c}_t^i$ and regret vector $\mathbf{r}_t^i$. The assignment belief vector is initialized to a uniform distribution, $\mathbf{a}_t^i \leftarrow [1/N_q, \dots, 1/N_q]^\top$, representing maximum uncertainty.

Step 2: The solver executes an iterative loop where robots exchange information and update beliefs. In each iteration k :

- 1) **Factor Linearization:** Each robot linearizes the measurement functions of its connected factors, e.g., the collision avoidance factor h^{ctij} (13) and assignment factor h^a (14), linearized around the current belief $\mathbf{a}_t^i$.
- 2) **Message Computation:** Robots compute outgoing messages to their neighbors. For the collision avoidance factor, robot i sends its current projected preference π_t^i to robot j . This allows robot j to evaluate the match score κ_t^{ij} (11) and determine the collision penalty.
- 3) **Belief Update:** Robots aggregate incoming messages from all factors (unary: cost, regret; pairwise: collision avoidance). The belief $\mathbf{a}_t^i$ is updated by minimizing the linearized squared error between the measurement functions and the zero-target observations ($\mathbf{z} = \mathbf{0}$). This pulls the belief toward a state that balances low individual cost with low inter-robot conflict.

Step 3: At the end of each iteration, the global convergence factor h^v is evaluated via a consensus check. The algorithm terminates when the conflict measure satisfies $v_t \leq \varepsilon$. Upon convergence, the discrete task assignment is extracted by applying the selection threshold τ .

V. APPLIED PROBLEMS: PE AND MRPD SCENARIOS

In this section, we utilize the factor graph detailed in Sec. IV to solve the task allocation problem in two well-known problems: PE and MRPD. In both problems, robots aim to navigate to the task while avoiding collisions with each other and obstacles. At each iteration, robots update their state estimates, perform task allocation based on the current estimates, and subsequently generate motion plans toward the assigned tasks.

A. Pursuit-Evasion

In this scenario, we consider a team of N_p pursuers and a team of N_q evaders operating in $\mathbb{R}^2$. If pursuers outnumber evaders ($N_p > N_q$), some pursuers will remain unassigned.

The pursuers aim to intercept the evaders to prevent them from reaching a predefined goal. An evader j is considered captured if the Euclidean distance between any pursuer i and the evader is less than a capture threshold d_c :

$$\exists i \in \{1, \dots, N_p\}, \exists j \in \{1, \dots, N_q\} : \|\mathbf{x}_t^{p_i} - \mathbf{x}_t^{q_j}\| \leq d_c, \quad (17)$$

where $\mathbf{x}_t^{p_i}$ and $\mathbf{x}_t^{q_j}$ denote the poses of the i^{th} pursuer and j^{th} evader at time t , respectively, see Table IV.

The PE problem is solved by integrating the factor graph shown in Fig. 2 to estimate evader poses and plan collision-free pursuer trajectories. A dedicated task allocation factor graph ensures a one-to-one assignment between pursuers and evaders. Beyond the coordination variables, we incorporate kinematic factors representing the motion of both pursuers and evaders, denoted by $f_t^{dp_i}$ and $f_t^{dq_i}$, respectively. Specifically, $f_t^{dp_i}$ characterizes the motion model of the i^{th} pursuer between time $t-1$ and t , while $f_t^{dq_i}$ characterizes the i^{th} evader's motion over the same interval, as detailed in Sec. V-C.

B. Package Pickup and Delivery

In the MRPD scenario, a set of N_p robots is tasked with transporting N_q packages from pickup locations to designated destinations. Each robot i is characterized by a carrying capacity L^i , while each package j has a specific weight W^j . A feasible assignment is strictly defined by the capacity constraint $L^i \geq W^j$.

Estimation and planning are performed using the factor graph framework shown in Fig. 2. Since the packages are static targets, the factor graph excludes kinematic task factors, distinguishing this from the PE problem. To enforce the feasibility of assignments, the task evaluation factor incorporates a capacity-aware regularization term. This term accounts for both the Euclidean distance to the task and the hard constraint $L^i \geq W^j$, ensuring that robots are only assigned to packages within their physical capabilities.

C. Factor Formulations

To implement GBPTA for the PE and MRPD scenarios, seven factor types are used. These are organized into four groups: kinematic, measurement, avoidance, and planning that are mostly inherited from Robot Web [18] and GBPlanner [19].

1) Kinematic Factors

The kinematic factors model the relative displacement between consecutive poses of the robots and dynamic tasks [22]. The robot kinematic factor is denoted by $f_t^{dp}(\mathbf{x}_t^{p_i}, \mathbf{x}_{t-1}^{p_i})$, while the factor for moving tasks is represented by $f_t^{dq}(\mathbf{x}_t^{q_j}, \mathbf{x}_{t-1}^{q_j})$.

These factors are implemented as Gaussian likelihoods where the cost is derived from the motion residual. The observation functions for the task and robot kinematics are:

$$h_t^{dq_j}(\mathbf{x}_t^{q_j}, \mathbf{x}_{t-1}^{q_j}) = \|\mathbf{x}_t^{q_j} \ominus \mathbf{x}_{t-1}^{q_j}\|_{\Omega_t^{q_j}}^2, \quad (18)$$

$$h_t^{dp_i}(\mathbf{x}_t^{p_i}, \mathbf{x}_{t-1}^{p_i}) = \|\mathbf{x}_t^{p_i} \ominus \mathbf{x}_{t-1}^{p_i}\|_{\Omega_t^{p_i}}^2, \quad (19)$$

where $\|\cdot\|_{\Omega}$ is the Mahalanobis distance weighted by the information matrix Ω . The $\ominus$ operator computes the pose difference on the $SE(2)$ manifold [23].

2) Measurement Factors

Measurement factors model the relative constraints between robots, tasks, and fixed landmarks [22]. The measurement factor between robot i and task j is denoted by $f_t^{mp_{ij}}$, while the factor between robot i and landmark j is represented by $f_t^{mo_{ij}}$. The associated observation functions are defined as:

$$h_t^{mp_{ij}}(\mathbf{x}_t^{q_j}, \mathbf{x}_t^{p_i}) = \|\mathbf{z}_t^{q_j} \ominus g(\mathbf{x}_t^{q_j}, \mathbf{x}_t^{p_i})\|_{\Omega_t^{p_{ij}}}^2, \quad (20)$$

$$h_t^{mo_{ij}}(\mathbf{x}_t^o, \mathbf{x}_t^{p_i}) = \|\mathbf{z}_t^o \ominus g(\mathbf{x}_t^o, \mathbf{x}_t^{p_i})\|_{\Omega_t^{p_o}}^2, \quad (21)$$

where $g(\cdot)$ denotes the sensor observation model, $\mathbf{z}$ represents the received measurement, and $\mathbf{x}$ denotes the estimated poses. The cost $h_t^{mp_{ij}}$ penalizes the discrepancy between the expected relative pose of the j^{th} task and its actual observation by the i^{th} robot, while $h_t^{mo_{ij}}$ enforces consistency with static landmarks $\mathbf{x}_j^o$ to bound global estimation error.

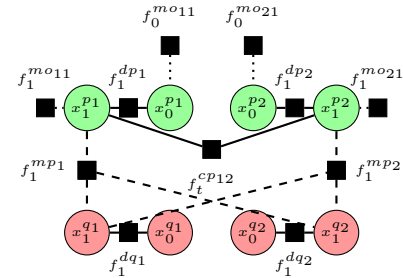


Fig. 2. The factor graph representation for the PE problem or any problem that involves moving tasks. Green nodes represent the variables associated with the poses of the pursuers. Red nodes are poses of evaders. Squares represent the constraints between the variables, including motion models (e.g. f_1^{dp1}), measurements of the robots (e.g. f_1^{mo11}) or landmarks (e.g. f_1^{mo11}), and robot collision avoidance factor (e.g. f_t^{cp12}).

3) Planning Factor

The planning factor, denoted by $f_t^{mopj}(\mathbf{x}_t^{q_j}, \mathbf{x}_t^{p_i})$, generates a path for the robots toward their assigned task locations. Each robot i attempts to minimize the distance to the estimated pose of task j , denoted by $\mathbf{x}_t^{q_j}$. The cost associated with this planning factor is [19]:

$$h_t^{mopj}(\mathbf{x}_t^{q_j}, \mathbf{x}_t^{p_i}) = \|\mathbf{x}_t^{q_j} \ominus \mathbf{x}_t^{p_i}\|_{\Omega_t^{p_{ij}}}^2, \quad (22)$$

where the residual is the relative pose difference on the manifold.

4) Avoidance Factors

The avoidance factors are designed to prevent collisions with other robots and environmental obstacles. To maintain a safety margin of radius d_s between robots, the robot collision avoidance factor $f_t^{cpij}(\mathbf{x}_t^{p_i}, \mathbf{x}_t^{p_j})$ is defined. The associated cost function, h_t^{cpij} is [19]:

$$h_t^{cpij}(\mathbf{x}_t^{p_i}, \mathbf{x}_t^{p_j}) = \begin{cases} \left\| 1 - \frac{d_p}{c_1} \right\|_{\Omega_t^p}^2 & \text{if } d_p < d_s \\ 0 & \text{if } d_p \geq d_s \end{cases}, \quad (23)$$

where $d_p = \|\mathbf{x}_t^{p_i} - \mathbf{x}_t^{p_j}\|$ represents the inter-robot distance.

Similarly, an obstacle avoidance factor $f_t^{opik}(\mathbf{x}_t^{p_i}, \mathbf{x}_t^{o_k})$ is defined to maintain a safety bubble of radius d_{so} around obstacles. The corresponding cost function, h_t^{opik} , is given by:

$$h_t^{opik}(\mathbf{x}_t^{p_i}) = \begin{cases} \left\| 1 - \frac{d_o}{c_2} \right\|_{\Omega_t^o}^2 & \text{if } d_o < d_{so} \\ 0 & \text{if } d_o \geq d_{so} \end{cases}, \quad (24)$$

where d_o is the distance between the i^{th} robot and the k^{th} obstacle, and c_1, c_2 are scaling constants used to tune the penalty gradient.

VI. EXPERIMENTS

This section evaluates GBPTA via simulation and real-world trials, comparing it against baselines across various conditions. Simulations were developed in Python using Pygame [24]. Hardware tests utilized an Intel i7-11700K CPU (8 cores, 16 threads). An interactive demo for PE and MRPD scenarios is available at: <https://gbpta.github.io/demo>.

GBPTA was evaluated across diverse scenarios, including PE with dynamic trajectories (Fig. 3). In a PE case with 30 pursuers (green circles) and 30 evaders (red circles), robots traveled an average of 182 m, with a maximum individual distance of 271 m.

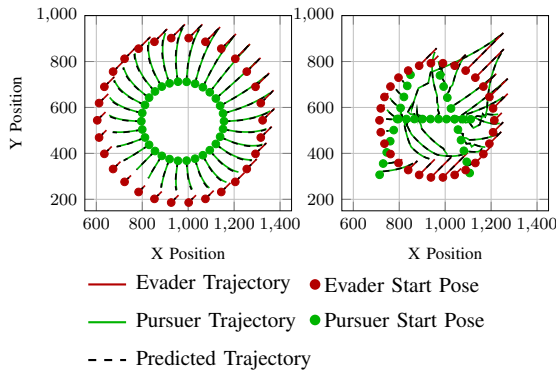


Fig. 3. Comparison of trajectories in two PE scenarios. In both experiments, there are 30 robots and 30 tasks.

A. Comparison with State-of-the-Art Methods

GBPTA was benchmarked against distributed and centralized methods over 100 simulations with four robots and four tasks in a 1920×1080 pixel environment. Performance was measured via the average total robot-task distance:

$$D_{\text{avg}} = \frac{1}{N} \sum_{k=1}^N \sum_{i=1}^{N_p} \|\mathbf{x}^{p_i} - \mathbf{x}^{q_{a_i}}\|, \quad (25)$$

where $N = 100$ runs and $\mathbf{x}^{q_{a_i}}$ is the task assigned to robot i . It is considered that initial robot and task positions are known and not estimated.

TABLE I
AVERAGE AND STANDARD DEVIATION OF TOTAL DISTANCE AND COMPUTE TIME FOR DIFFERENT APPROACHES. BLUE INDICATES THE BEST PERFORMANCE AMONG DISTRIBUTED METHODS, WHILE GREEN DENOTES THE OVERALL BEST RESULTS ACROSS ALL APPROACHES, INCLUDING CENTRALIZED.

Type	Method	Avg Distance (m)↓	Computation Time (ms)↓
Distributed	GBPTA (ours)	1458 ± 423	2.33 ± 1.04
	Dist. Hungarian [11]	1917 ± 568	2.42 ± 0.05
	ADMM [25]	2119 ± 532	2.34 ± 0.15
	CBBA [12]	1506 ± 442	5.95 ± 1.22
Centralized	PDAGA [7]	1420 ± 420	61.5 ± 0.87
	Hungarian [6]	1419 ± 420	0.9 ± 7.7
	MRMTA [9]	2265 ± 482	0.2 ± 0.02
	Market-based [26]	1531 ± 448	31 ± 4.6
	Hybrid SUS [8]	1467 ± 498	836 ± 123

As shown in Table I, GBPTA achieves the lowest distance among non-centralized approaches (blue highlights), that is 23.95% improvement over distributed Hungarian and 3.18% over CBBA. It is important to highlight that while CBBA requires every agent to maintain a complete copy of the Winning Bids List for all tasks in the system, making it a decentralized method. This global data requirement can be seen as a limitation in truly large-scale distributed environments. In contrast, GBPTA is a fully distributed method that achieves coordination through local message passing, avoiding the need for each agent to store the global assignment state. GBPTA also remains within 3% of the centralized Hungarian [6] and PDAGA [7] methods (green highlights). While the Hungarian algorithm is faster, it is centralized and assumes perfect information. Critically, GBPTA is the only approach that accounts for measurement uncertainty and avoids single-point failures.

Task execution efficiency was evaluated utilizing Makespan [27]. Makespan is the total time required to complete a set of tasks, measured from the start of the first task to the completion of the last task. Over 100 simulation runs with different numbers of robots and packages, we compare GBPTA with the centralized Hungarian algorithm, a distributed version of Sequential Greedy task allocation [10], and DANCERS [20]. In the distributed Sequential Greedy approach, robots select tasks individually in a sequential manner, communicate their decisions to their peers, and the remaining robots choose from the remaining tasks. As shown in Fig. 4, GBPTA performs better than Sequential Greedy and DANCERS. On average, it reduces the Makespan by 10% and 22.3% in comparison to Sequential Greedy when there are 4 and 8 robots in the scene, respectively. It also reduces the Makespan by 32% and 71.57% in comparison to DANCERS when there are 4 and 8 robots in the scene respectively.

To evaluate the performance of GBPTA in large-scale scenarios, we compare it with CBBA, another distributed task allocation algorithm that achieves performance comparable to GBPTA. In this experiment, robots are allowed to communicate only with neighboring robots within their communication range. The communication radius is set to 100 ($c_r = 100$). As

shown in Fig. 5, GBPTA consistently achieves the lowest total travel distance. It outperforms CBBA by 6.2%, 3.9%, 4.3%, and 0.6% and CBBA with noisy communication by 7.2%, 4.2%, 10.7%, and 4.3% for 8, 16, 32, and 64 robots and tasks, respectively. The total travel distance increases with team size for all methods. To ensure a fair comparison, task allocation is performed using noisy measurements. We also include a modified version of CBBA that accounts for communication noise, similar to GBPTA.

B. Feature Scalability and Energy Efficiency

GBPTA scales effectively as additional task attributes, such as robot capacity, energy levels, and package weights, are integrated into the factor graph. We evaluated this by comparing distance-only allocation against a multi-feature model (remaining energy, weight of packages, capacity of robots). As shown in Fig. 6, incorporating these constraints significantly reduces total energy consumption. As shown in Fig. 6, over 100 trials, the energy usage decreases when robot energy, package weight, and robot capacity are considered in task allocation, with reductions of 6.4% and 20% observed for 4 and 8 robots, respectively, in 100 simulated experiments.

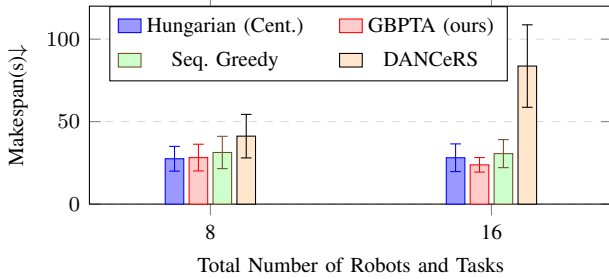


Fig. 4. Comparison of Makespan for different numbers of robots and packages in package pickup and delivery scenario using DANCeRS, GBPTA, Sequential Greedy, and Hungarian methods. Results are averaged over 100 simulation runs, where robots are randomly placed in the scene. GBPTA reduces Makespan in comparison to distributed methods for different number of robots and tasks.

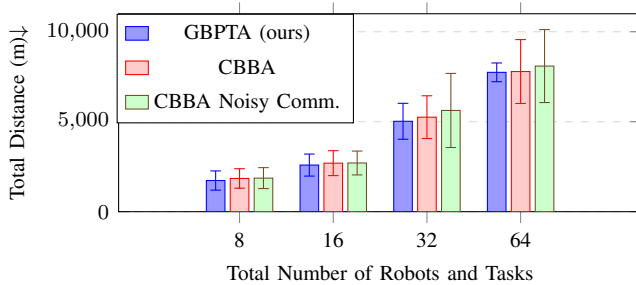


Fig. 5. Comparison of the total distance traveled by all robots for different numbers of pursuers and evaders using GBPTA, CBBA without noisy communication, and CBBA with noisy communication over 100 simulation trials. Error bars indicate the standard deviation. Task allocation is performed based on noisy estimated values. Overall, GBPTA achieves the lowest total travel distance across all team sizes.

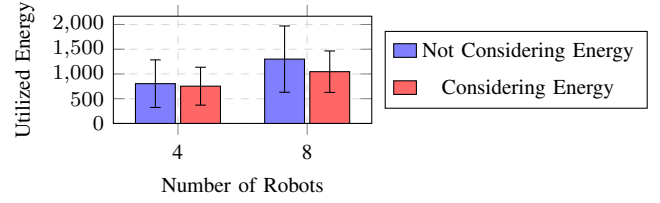


Fig. 6. Energy usage reduction for different numbers of robots in MRPD, considering the remaining energy of the robots, the weight of the packages, the capacity of the robots and distance to tasks. The weights of packages and the capacities of robots are assumed to range between 25 and 50 kilograms. The red bar shows the results when, in addition to distance, robot capacity, and package weight, the robots' remaining energy is also considered. It shows that when the remaining energy is considered, the robots' energy consumption decreases and GBPTA can consider multiple features while allocating tasks.

C. Performance Under Realistic Communication Degradation

In this section, the impact of packet loss, asynchronous message passing, and message delays on GBPTA performance is presented. For packet loss, Makespan is evaluated for four robots and four packages under varying message loss rates. Results are averaged over 100 simulation runs, with each run consisting of 20 iterations per timestep. As shown in Fig. 7, GBPTA remains robust even when up to 30% of messages are dropped, exhibiting only a modest increase in makespan.

Over 100 simulation runs, the effects of asynchronous message passing and delayed messages with four pursuers and four evaders are shown in Fig. 8. The “No Delay” case represents the scenario in which message passing is synchronized, while asynchronous message passing is represented by the dashed bar. As shown in Fig. 8, the total distance between robots and their assigned tasks increases by 7% under asynchronous message passing. The the total distance between robots and their assigned tasks increases by 1.6%, 3.8%, and 6.5% when one robot sends its messages after 5, 10, and 15 seconds, respectively.

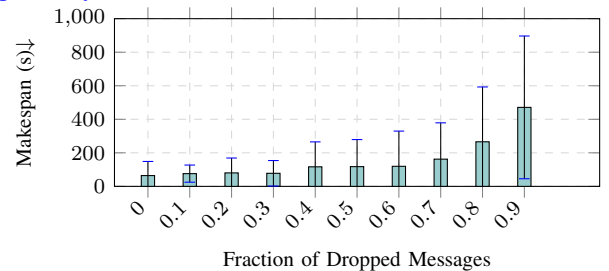


Fig. 7. Effect of the message drop fraction on the makespan. Results are averaged over 100 simulation runs with four robots and four packages. GBPTA remains robust even when up to 30% of messages are dropped.

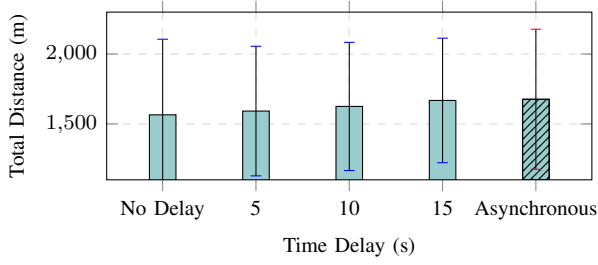


Fig. 8. Effect of message delays on the average utility when there are four pursuers and four evaders in the scene. The “No Delay” case represents the baseline without any delays, while the dashed bar represents asynchronous message passing. Error bars indicate the standard deviation. Results are averaged over 100 simulation runs. GBPTA is able to maintain a similar assignment utility when robots send messages with delays or when message passing is asynchronous, with only a slight increase in utility compared with the synchronous case.

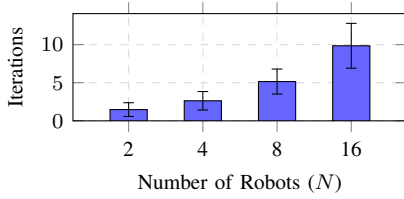


Fig. 9. Average number of required iterations that are required for converging to a solution over 100 simulation runs.

D. Time Complexity and Convergence

For specific structures and a limited number of robots and tasks, the time complexity is $O(N^2)$. As shown in Fig. 9, in the worst case, N iterations of message passing are needed [28], where N is the number of robots and tasks. This matches the performance of the distributed Hungarian algorithm but provides a better alternative to both the original Hungarian method and CBBA, both of which scale cubically at $O(N^3)$ [12].

TABLE II
SUM OF THE CONVERGENCE VALUES $\sum_{i=1}^{N_p} \sum_{j=i+1}^{N_p} v_t^{ij}$ FOR DIFFERENT PENALTY SCALES p AND SMOOTHNESS-CONTROL PARAMETERS β AT ITERATIONS 10, 20, AND 30 FOR A SCENE CONTAINING FOUR PURSUERS AND FOUR EVADERS. DARKER GREEN INDICATES A LOWER SUM OF THE CONVERGENCE VALUES.

Iteration	p	$\beta = 10^{-2}$	$\beta = 1$	$\beta = 10^2$
10	10^2	5.09 ± 3	5.10 ± 3	5.10 ± 3
	10^4	5.6 ± 3.28	2.00 ± 1.45	1.4 ± 1.31
	10^6	2.4 ± 1.66	0.79 ± 1.19	0.30 ± 0.73
20	10^2	5.1 ± 3	4.7 ± 3.13	4.89 ± 3.07
	10^4	5.5 ± 3.63	1.79 ± 1.10	0.8 ± 1
	10^6	2.1 ± 1.37	0 ± 0	0 ± 0
30	10^2	4.39 ± 3.15	4.79 ± 3.20	4.69 ± 3.13
	10^4	5.3 ± 3.45	1.49 ± 1.43	0 ± 0
	10^6	1.99 ± 1.99	0 ± 0	0 ± 0

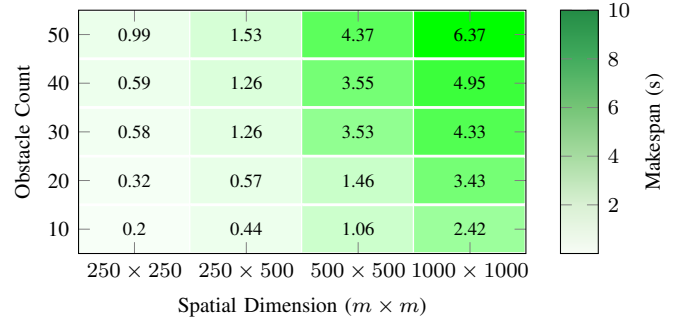


Fig. 10. Makespan comparison under different numbers of obstacles and scene dimensions, which may result in dense, loopy factor graphs. There are eight robots and eight packages in the scene. The results are averaged over 100 simulation runs. Dimensions are given in meters. Overall, the results show that GBPTA maintains consistent makespan performance as the number of obstacles and the scene dimension increase, although the makespan also increases.

Different hyperparameters and factors affect the performance of the proposed solution. To investigate the effects of the penalty scale p in Eq. 13 and β in Eqs. 12 and 15, we evaluate the sum of the convergence values, $\sum_{i=1}^{N_p} \sum_{j=i+1}^{N_p} v_t^{ij}$, over the iterations for a task allocation problem with four pursuers and four evaders. The experiments are conducted over 100 simulation runs. As shown in Table II, increasing the penalty scale p from 10^2 to 10^6 generally leads to faster convergence. Increasing β also generally accelerates convergence. Removing the Task Collision Avoidance Factor ($f_t^{ct_{ij}}$) and using $\epsilon \geq 1/N_p$ in $f_t^{v_{ij}}$ can cause duplicate task assignments.

Although convergence of GBP in loopy graphs is not guaranteed, the results in Fig. 10 demonstrate that the proposed approach can successfully complete the assigned tasks and deliver all eight packages using eight robots, even in a limited space $250\text{ m} \times 250\text{ m}$ workspace with a large number of obstacles. Each configuration is evaluated over 100 simulation runs. The makespan increases from 0.20 s with 10 obstacles to 0.99 s with 50 obstacles for the $250\text{ m} \times 250\text{ m}$ workspace, indicating that the proposed approach remains effective as the obstacle density increases. Yates et al. [29] show that, under four conditions, the variables in the graph converge to Gaussian distributions.

E. Real-World Experiments

To validate GBPTA’s real-world applicability and compare it with DANCeRS, we implemented an 8-vs.-8 scenario in which robots navigate using noisy odometry data, assuming that only their initial positions are known in the presence of obstacles. GBPTA successfully yields a near-optimal assignment. Although the tasks are positioned such that multiple robots may prefer the same task GBPTA is able to achieve a one-to-one assignment. As shown in Fig. 11, starting from their initial positions (black circles), the robots successfully coordinate their trajectories to intercept their respective targets without colliding with obstacles, demonstrating the framework’s robustness to sensor noise and real-time communication constraints. The robots are able to reach their assigned tasks within 63 seconds; however, DANCeRS is unable to achieve a one-to-one assignment, with only four tasks completed within the

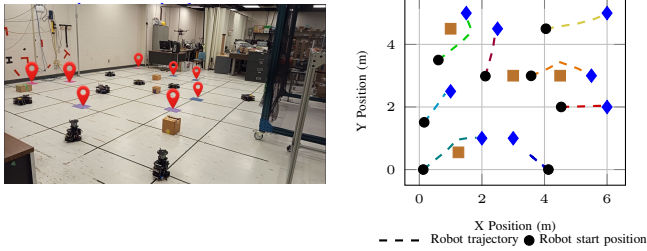


Fig. 11. Performance of GBPTA in a real-world scenario. Left: the physical environment; right: the resulting trajectories, where black circles represent the initial poses. In the real-world image, tasks are represented by pink and blue squares, while in the plot, they are represented by diamonds. Obstacles are represented by brown squares. The robots achieve a one-to-one assignment in a distributed manner and reach their respective goals without colliding with obstacles in the scene. Video link: <https://gbpta.github.io/demo>

VII. CONCLUSION

This paper presented GBPTA, a Gaussian Belief Propagation-based framework for distributed multi-robot task allocation, capable of integration with tracking and planning. By formulating coordination as a factor graph, GBPTA handles measurement uncertainty and scales effectively across various robot-task densities. We demonstrated the convergence of the algorithm through various experiments. Evaluations in PE and Pickup and Delivery Problem (PDP)MRPD scenarios, through both simulations and ROS2-based hardware trials, show that GBPTA outperforms existing distributed baselines and achieves near-optimal performance within 3% of centralized methods. Future work will explore integrating Graph Neural Networks with GBP to further enhance coordination logic and solve complex allocation problems in highly dynamic environments.

REFERENCES

- [1] H. Yang, P. Ge, J. Cao, Y. Yang, and Y. Liu, "Large scale pursuit-evasion under collision avoidance using deep reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2023, pp. 2232–2239.
- [2] J. Hooks, M. S. Ahn, J. Yu, X. Zhang, T. Zhu, H. Chae, and D. Hong, "ALPHRED: A multi-modal operations quadruped robot for package delivery applications," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 5409–5416, 2020.
- [3] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, D. Sadigh, and J. Bohg, "Dynamic multi-robot task allocation under uncertainty and temporal constraints," *Auton. Robot.*, vol. 46, no. 1, pp. 231–247, 2022.
- [4] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, "Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?" in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 4311–4317.
- [5] Z. Feng, G. Hu, Y. Sun, and J. Soon, "An overview of collaborative robotic manipulation in multi-robot systems," *Annu. Rev. Control.*, vol. 49, pp. 113–127, 2020.
- [6] H. W. Kuhn, "The Hungarian method for the assignment problem," *Nav. Res. Logist. Q.*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [7] W. Lu, W. Gao, B. Liu, W. Niu, X. Peng, Z. Yang, and Y. Song, "Parallel dual adaptive genetic algorithm: A method for satellite constellation task assignment in time-sensitive target tracking," *Adv. Space Res.*, vol. 74, no. 10, pp. 5192–5213, 2024.
- [8] S. Chen, T. X. Lin, S. Al-Abri, R. C. Arkin, and F. Zhang, "Hybrid SUSD-based task allocation for heterogeneous multi-robot teams," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 1400–1406.
- [9] H. Osooli, P. Robinette, K. Jerath, and S. R. Ahmadzadeh, "A multi-robot task assignment framework for search and rescue with heterogeneous teams," *arXiv:2309.12589 [cs.RO]*, 2023.
- [10] K. Suresh, A. Rauniyar, M. Corah, and S. Scherer, "Greedy perspectives: Multi-drone view planning for collaborative perception in cluttered environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2024, pp. 10990–10997.
- [11] S. Chopra, G. Notarstefano, M. Rice, and M. Egerstedt, "A distributed version of the Hungarian method for multi-robot assignment," *IEEE Trans. Robot.*, vol. 33, no. 4, pp. 932–947, 2017.
- [12] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Trans. Robot.*, vol. 25, no. 4, pp. 912–926, 2009.
- [13] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2011.
- [14] B. Li and H. Ma, "Double-deck multi-agent pickup and delivery: Multi-robot rearrangement in large-scale warehouses," *IEEE Robot. Autom. Lett.*, vol. 8, no. 6, pp. 3701–3708, 2023.
- [15] A. Camisa, A. Testa, and G. Notarstefano, "Multi-robot pickup and delivery via distributed resource allocation," *IEEE Trans. Robot.*, vol. 39, no. 2, pp. 1106–1118, 2022.
- [16] X. Bai, A. Fielbaum, M. Kronmüller, L. Knoedler, and J. Alonso-Mora, "Group-based distributed auction algorithms for multi-robot task assignment," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 2, pp. 1292–1303, 2022.
- [17] A. J. Davison and J. Ortiz, "FutureMapping 2: Gaussian Belief Propagation for Spatial AI," 2022.
- [18] R. Murai, J. Ortiz, S. Saeedi, P. H. J. Kelly, and A. J. Davison, "A robot web for distributed many-device localization," *IEEE Trans. Robot.*, vol. 40, pp. 121–138, 2023.
- [19] A. Patwardhan, R. Murai, and A. J. Davison, "Distributing collaborative multi-robot planning with Gaussian belief propagation," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 552–559, 2022.
- [20] A. Patwardhan and A. J. Davison, "DANCERS: A distributed algorithm for negotiating consensus in robot swarms," in *IEEE Int. Conf. Robot. Autom.*, 2026.
- [21] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [22] F. Dellaert and M. Kaess, *Factor Graphs for Robot Perception*. Now Publishers Inc., 2017, vol. 6, no. 1–2.
- [23] R. Smith, M. Self, and P. Cheeseman, "Estimating uncertain spatial relationships in robotics," in *Autonomous Robot Vehicles*, I. J. Cox and G. T. Wilfong, Eds. Springer, 1990, pp. 167–193.
- [24] Pygame Developers, "Pygame," [Online]. Available: <https://www.pygame.org>, accessed: Nov. 23, 2025.
- [25] N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2011.
- [26] M. Otte, M. Kuhlman, and D. Sofge, "Multi-robot task allocation with auctions in harsh communication environments," in *Proc. IEEE Int. Symp. Multi-Robot Multi-Agent Syst. (MRS)*, 2017, pp. 32–39.
- [27] M. Pinedo and K. Hadavi, "Scheduling: Theory, algorithms and systems development," in *Proc. Oper. Res. 1991*. Springer, 1992, pp. 35–42.
- [28] M. Argyle, D. W. Casbeer, and R. Beard, "A multi-team extension of the consensus-based bundle algorithm," in *Proc. IEEE American Control Conference (ACC)*, 2011, pp. 5376–5381.
- [29] T. Yates, Y. Cheng, I. Alzugaray, D. Akarca, P. A. Mediano, and A. J. Davison, "Belief propagation converges to gaussian distributions in sparsely-connected factor graphs," *arXiv preprint arXiv:2601.21935*, 2026.